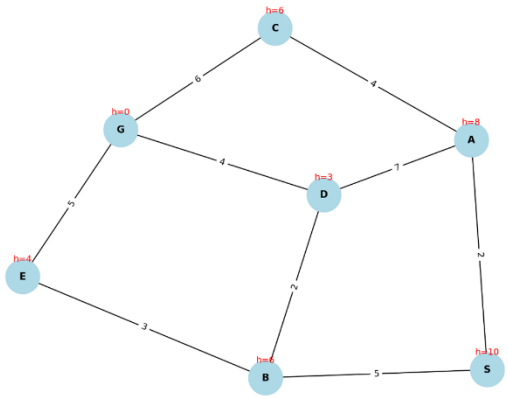
Assignment 2

Search algorithm, Genetic Algorithm, Data Science and Implementations

Problem search algorithm

A delivery robot must travel between locations in a country. The map is represented as a weighted graph where nodes are cities and edges represent travel costs (in kilometers). The goal is that you will design an agent using A* search algorithm to find the cheapest path from Start (S) to Goal (G). The graph that we assume has some heuristics distances for reaching the target node, the heuristic distances for reaching G are written in red the distance between single nodes are in black.



Heuristic distances: in this case of a minimum problem the value of the heuristics is always to be lower or equal to the real value of the distances. For example S has real distance 11 and the heuristic distance is 10, A node has a real distance 10 and the heuristic says 8 that is admissible for the heuristics. The useful meaning of the heuristic distances is that they should be easy to calculate without the application of the BFS or DFS in that case we can describe an algorithm that don't use heuristics.

Problem 1 part A

Describe an algorithm to find the cheapest path from S to G. The algorithm that we can design for the agent could be the follow just thanks to the heuristics distance that enables us to base the tour just by visiting the neighbors and their distances for reaching the target. Instead of visiting all the possible paths from each node.

Input of the graph-> evaluation of the heuristic's distances -> Apply of the Agent A*

Algorithm for agent A*:

The algorithm is capable of return the path for visiting the node. The algorithm should receive the graph with the heuristics for each node. The path in that case is a list.
Complexity of algorithm O(G(V)), the algorithm should just visit each node once.

- Initialize the queue
- While target not reached:
  - Verify if the target node is reached, in that case the solution is found and return the queue
  - Visit the neighbors
  - Select the next one node to input in the path by choosing his heuristic distances and put in append of the queue

Problem 1 part B

This problem is polynomial If the graph is not huge, and we can apply a Binary PLI that as a result should give us which node visit after S for reach the target G

Variables

i and j are nodes $\in G$

$$Y_{ij} = \begin{cases} 1 & \text{if the path includes nodes } i \text{ and } j \text{ select the edge} \\ 0 & \end{cases}$$

$W_{ij} \in \mathbb{N}$ Euristics weight of the edges (it work also with the real values)

Cost Function

$$\begin{cases} \min \sum_{i \in V} \sum_{j \in V / \{i\}} Y_{ij} W_{ij} \\ \sum_{j \in G(V)/\{A\}} Y_{Aj} \geq 1 \\ \sum_{i \in G(V)/\{G\}} Y_{iG} \geq 1 \end{cases}$$

$G(V) =$

$i = \{A, B, C, D, E, S, G\}$

$j = \{A, B, C, D, E, S, G\}$

Final solution

$S \rightarrow B \rightarrow D \rightarrow G$

Cost Estimated B cost 10

Real Cost 11

Genetic algorithm

Solve a maze problem using GA. The aim of this task is to find a path in a maze, shown in Figure 2, for a mouse to reach the destination or food with the shortest and acceptable path. You are going to implement a Genetic Algorithm that will find a solution to find a path by optimizing the problem automatically.

This exercise was so challenging to me for the following three problems:

- How to model the input problem, usually the maze problem have block instead of line walls
- What to write in the chromosome?
- How to build the cost functions
- What efficient solution can be applied?

Build the model for the received input

I had to transpose the image to a matrix that could fit the problem

Maze from assignment:

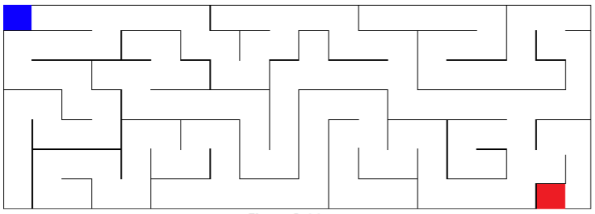
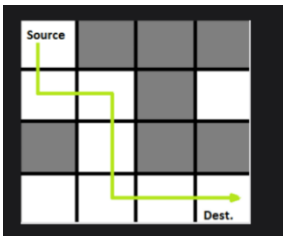


Figure 2. Maze.

Maze from the classical maze:



Solution:

- If the starting matrix was MxN with M rows and N columns the newer matrix is going to be M*2xN*2
- First put the cell in the even indices (2*i+1, 2*j+1), in every of these cells put one and they reflect the cells of the original matrix
- Put the wall in the odd indices, 0 if there is a wall 1 if there isn't a wall and the rat can pass to the following cells
- Fill the cells in the 0 in the odd rows because the rat cant move along the diagonal

Matrix in excel :

Real indices		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38
	Indices of the cells	0		1		2		3		4		5		6		7		8		9		10		11		12		13		14		15		16		17		18		19
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1
1		0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	1	0	1	0	0
2	1	1	1	1	1	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	0	1	1	1	1	1	1	0	1	1	1	1	1	0	1	1	1	0	1
3		1	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	1	0	0	0	1	0	0	0	0	0	1	0	1	0	0	0	0	0	1	0	0	0	1
4	2	1	1	1	1	1	1	1	1	1	1	1	0	1	0	1	1	1	0	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	0	1
5		0	0	0	0	1	0	1	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1
6	3	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1
7		1	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	1	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0
8	4	1	0	1	1	1	1	1	0	1	1	1	0	1	1	1	0	1	0	1	0	1	0	1	1	1	0	1	1	1	0	1	1	1	1	1	0	1	1	1
9		1	0	0	0	0	0	0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	0	1	0	1	0	1
10	5	1	0	1	1	1	1	1	0	1	0	1	1	1	0	1	1	1	1	1	0	1	0	1	0	1	1	1	0	1	0	1	1	1	0	1	1	1	0	1
11		1	0	1	0	0	0	1	0	1	0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	1
12	6	1	0	1	1	1	0	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	0	1	1	1	1	1	1	1	0	1	1	1

	0		1		2		3		4		5		6		7		8		9		10		11		12		13		14		15		16		17		18		19
0	1	1	1	1	1	1	1	1	1	1	1	1	1		1	1	1	1	1	1	1	1	1		1	1	1	1	1	1	1	1	1		1	1	1	1	1
							1						1						1				1										1		1		1		
1	1	1	1	1	1	1	1		1	1	1		1	1	1		1	1	1		1		1	1	1	1	1		1	1	1	1	1		1	1	1		1
	1										1				1		1				1								1						1				1
2	1	1	1	1	1	1	1	1	1	1	1		1		1	1	1		1	1	1	1	1	1	1	1	1		1	1	1	1	1	1	1	1	1		1
				1		1		1											1								1												1
3	1	1	1		1	1	1	1	1	1	1	1	1	1	1	1	1		1		1	1	1	1	1		1	1	1	1	1	1	1	1	1	1	1	1	1
	1		1				1												1		1				1										1				
4	1		1	1	1	1	1		1	1	1		1	1	1		1		1		1		1	1	1		1	1	1		1	1	1	1	1		1	1	1
	1								1		1		1		1		1		1		1		1		1		1		1		1				1		1		1
5	1		1	1	1	1	1		1		1	1	1		1	1	1	1	1		1		1		1	1	1		1		1	1	1		1	1	1		1
	1		1				1		1						1						1		1						1						1				1
6	1		1	1	1		1	1	1		1	1	1	1	1	1	1	1	1	1	1		1	1	1	1	1		1	1	1	1	1	1	1		1	1	1

What to write in the chromosome?

I choose the sequence of moves for the chromosome and then build a path based on them.

The only position of the mouse can move in the maze matrices could be Up Down Right and Left:

- Right from (i,j) to (i,j+1)
- Left from (i,j) to (i,j-1)
- Down from (i,j) to (i+1,j+1)
- Up from (i,j) to (i-1,j)

The sequence of moves increases generation by generation for make them converge to the right direction in less moves and every 100 generation are inserted into the population new members to reduce the similarity between the vectors in the population and don't make the algorithm to maintain the same chromosomes every time. The add of new members happen in 2 ways: random or by simple put start/destination that must contains also a random point during the path. The crossover take the 2 selected parents' chromosome and combines them in a one point of joint. The mutation if in a position of the chromosome the mutation criteria is meet the chromosome can add from 2 to 4 new mutations that are random moves.

Fitness Function:

- Based on the 3 types of penalty: Hit walls / Out of the matrix / Cycles
- Boost for: moves from the start/ getting closer to the target/ explore/ long path and short seq of moves / Reach the destination with less sequence of nodes in the path and less hit on the walls

Added improvement:

- 2 approaches for generating a new member of the population
- Removed cycle in the path
- Percentage of mutation increases during the algorithm
- Singleton class to maintain the best metrics
- Slow increase for the sequence of moves

- Adaptive fitness function based on the generation
- Stopper criteria for approval also the close solution reached, based on value of patience or similar best values, where every 100 generations if the solution doesn't change the algorithm stops.

NB also when the best metrics have a low value of the hit walls in the same generation is because the best metrics take the best value with low hit walls, the print in the main show the best value of the population with the high score of fitness. Example :

Best chromosome hit 12 walls and the best metrics hit 8 during the 1100-1200 gen:

Gen: 1100 Solution:
(0,0) -> (1,0) -> (1,1) -> (1,2) -> (1,3) -> (2,3) -> (2,4) -> (3,4) -> (4,4) -> (4,5) -> (4,6) -> (4,7) -> (4,8) -> (4,9) -> (4,10) -> (4,11) -> (4,12) -> (4,13) -> (4,14) -> (4,15) -> (4,16) -> (4,17) -> (4,18) -> (4,19) -> (4,20) -> (4,21) -> (4,22) -> (4,23) -> (4,24) -> (4,25) -> (4,26) -> (4,27) -> (4,28) -> (4,29) -> (4,30) -> (4,31) -> (4,32) -> (4,33) -> (4,34) -> (4,35) -> (4,36) -> (4,37) -> (4,38) -> (5,38) -> (6,38) -> (6,37) -> (6,36) -> (7,36) -> (8,36) -> (9,36) -> (10,36) -> (11,36) -> (12,36) Fitness Score: 2353.8524327989094 Hit walls for : 12 wall in the best metrics : 8

Wall In the best metrics changed during the 1200-1300 gen, new solution with 7 walls hit

actual n of generation: 1200 Solution:
(0,0) -> (0,1) -> (0,2) -> (0,3) -> (0,4) -> (0,5) -> (0,6) -> (0,7) -> (0,8) -> (0,9) -> (0,10) -> (0,11) -> (0,12) -> (0,13) -> (0,14) -> (0,15) -> (0,16) -> (0,17) -> (0,18) -> (0,19) -> (0,20) -> (0,21) -> (0,22) -> (0,23) -> (0,24) -> (0,25) -> (0,26) -> (0,27) -> (0,28) -> (0,29) -> (0,30) -> (0,31) -> (0,32) -> (0,33) -> (0,34) -> (0,35) -> (0,36) -> (1,36) -> (2,36) -> (3,36) -> (4,36) -> (5,36) -> (6,36) -> (7,36) -> (8,36) -> (9,36) -> (10,36) -> (11,36) -> (12,36)

Best chromosome hit 12 walls and the best metrics hit 7 during the 1200-1300 gen:

Gen: 1200 Solution: (0,0) -> (1,0) -> (1,1) -> (1,2) -> (1,3) -> (2,3) -> (2,4) -> (3,4) -> (4,4) -> (4,5) -> (4,6) -> (4,7) -> (4,8) -> (4,9) -> (4,10) -> (4,11) -> (4,12) -> (4,13) -> (4,14) -> (4,15) -> (4,16) -> (4,17) -> (4,18) -> (4,19) -> (4,20) -> (4,21) -> (4,22) -> (4,23) -> (4,24) -> (4,25) -> (4,26) -> (4,27) -> (4,28) -> (4,29) -> (4,30) -> (4,31) -> (4,32) -> (4,33) -> (4,34) -> (4,35) -> (4,36) -> (4,37) -> (4,38) -> (5,38) -> (6,38) -> (6,37) -> (6,36) -> (7,36) -> (8,36) -> (9,36) -> (10,36) -> (11,36) -> (12,36) Fitness Score: 2348.2458225191835 Hit walls for : 12 wall in the best metrics : 7

Improvement tested and deleted :

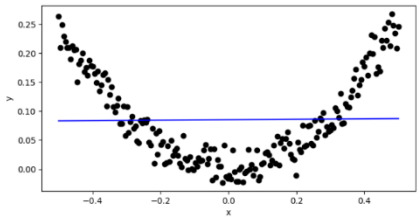
Some improvements were tested but then deleted because inefficient to converge at the solution of the maze:

- Reverse path that start from the end and reach the start for give different valid paths to the chromosomes, after the evaluation of the path make it reverse and take the distance from the start instead of the end-> no improvements
- Crossover that selects the best move to reach the target based on the distance-> the algorithm does not converge for decrease the penalties

- Crossover with a random selection of the movement from the sequence of the parents -> the algorithm does not converge to a solution high variance of the values of the penalties

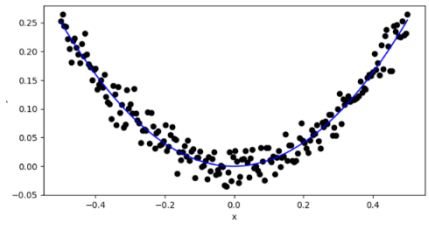
Data science and implementation

Task a:



I imported the linear regression model, trained the model, get the prediction from the trained model. Plotted the results.
The prediction are bad the model underfit the datapoints.

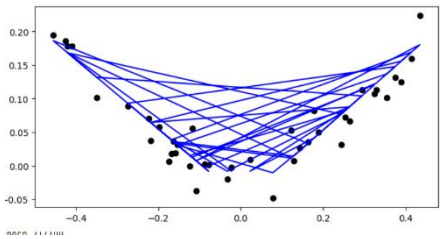
Task b:



Same process for the task a but this time I imported the polinomial regression. The prediction in that case are perfectly matched on the datapoints, the model learn the function of the datapoints.

Task c:

Split the data in training and testing, created the neural networks, the input layer is automatically defined when we define a dense neural network, for each features is created his neuron, then the data pass through a hidden layer with activation function Relu and at the end we get the linear prediction.
The Relu because it is the easiest function that is not linear and a linear activation function in the out layer. The neural network has to low neurons for the single layer and to less level for being trained properly on the polynomial dataset, so also if we apply a lot of epochs and the best optimizer for the training the prediction cant fit well the dataset input.



Task d:

I evaluated the models, the first 2 with all the dataset because we didn't have a split, the third one with the y_test because we splitted the data and also if the model are so different each other, the polynomial model seems to be the best but there isn't a large difference between the error of the other model.

```
Mean Squared Error of the linear model: 0.006005 of the polinomial model: 0.000409 of the ANN model: 0.000529
```

Task e:

The plot data is done in every section and show the differences between the real value and the predictions.